

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belagavi : 590018



A Mini-Project report on

DEEPFAKE AUDIO DETECTION USING DEEP LEARNING

Submitted in partial fulfillment of the requirement for the award of the degree of

Bachelor of Engineering
in
Computer Science and Engineering

by

DARSHAN M	1AY23CS065
GAJULA HARSHITH	1AY23CS077
M SHANKARLINGESWAR	1AY23CS107
SHAILAJA	1AY23CS166

Under the guidance of
Mrs. Shrutika C Rampure
Assistant professor



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
ACHARYA INSTITUTE OF TECHNOLOGY

(Affiliated to Visvesvaraya Technological University, Belgaum)

Accredited by NAAC and NBA

2025-2026

ACHARYA INSTITUTE OF TECHNOLOGY

Acharya Dr. Sarvepalli Radhakrishnan Road, Soladevanahalli, Bangalore – 560107
(Affiliated to Visvesvaraya Technological University, Belagavi)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



Certificate

Certified that the Mini-project (BCS586) entitled “**DEEPFAKE AUDIO DETECTION USING DEEP LEARNING**” is a Bonafide work carried out by **DARSHAN M (1AY23CS065),GAJULAHARSHITH(1AY23CS077),M SHANKARLINGESHWAR (1AY23CS107),SHAILAJA (1AY23CS166)** in partial fulfillment for the award of degree of **Bachelor of Engineering in Computer Science & Engineering of Visvesvaraya Technological University**, Belagavi during the year **2025-2026**. It is certified that all corrections/ suggestions indicated for internal assessments have been incorporated in the report deposited in the departmental library. This Mini-project (BCS586) report has been approved as it satisfies the academic requirements with respect to mini-project work(BCS586) prescribed for the **Bachelor of Engineering Degree**.

Signature of the Guide
Mrs. Shrutika C Rampure
Assistant professor

Signature of the HOD
Dr. Kala Venugopal
Associate professor

Signature of the Principal
Dr. C K Marigowda

Name of the Examiners

Signature with Date

1. _____

2. _____

DECLARATION

We the students of B.E. Computer Science & Engineering, Acharya Institute of Technology, Soladevanahalli, Bengaluru, hereby declare that the Mini-project work entitled “DEEPFAKE AUDIO DETECTION USING DEEP LEARNING” is an authentic work carried out and successfully completed under the supervision and guidance of Mr. Prashanth Kumar S P, Assistant Professor, Department of Computer Science and Engineering, Acharya Institute of Technology, Bengaluru. This dissertation work is submitted to Visvesvaraya Technological University in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in Computer Science and Engineering department during the academic year 2025-26. We have not submitted the matter embodied to any other university or institution for the award of any other degree.

Date:19/11/2025

Place: Bengaluru

NAME	USN	SIGNATURE
Darshan M	1AY23CS065	
Gajula Harshith	1AY23CS077	
M Shankarlingeswar	1AY23CS107	
Shailaja	1AY23CS166	

ACKNOWLEDGEMENT

We express our gratitude to our institution and management for providing us with good infrastructure, laboratory, facilities and inspiring staff, and whose gratitude was of immense help in completion of this report successfully.

We are deeply indebted to **Dr. C K Marigowda** Principal, Acharya Institute of Technology, Bengaluru, who has been a constant source of enthusiastic inspiration to steer us forward.

We sincerely thank our respected Dean of academics, **Dr. Rajeswari**, for her invaluable guidance, support, and encouragement throughout this Mini-project.

We heartily thank **Dr. Kala Venugopal**, Associate professor, Head of the Department, Department of Computer Science and Engineering, Acharya Institute of Technology Bengaluru, for her valuable support and for rendering us resources for carrying out this Mini-project work.

We specially thank **Mrs. Shrutika C Rampur**, Assistant professor, Department of Computer Science and Engineering who guided us with valuable suggestions in completing this Mini-project work at every stage.

Also, we wish to express deep sense of gratitude for project coordinators **Mr. Prashanth Kumar S P**, Assistant Professor and **Mr. Akash Raj**, Assistant Professor, Department of Computer Science and Engineering, Acharya Institute of Technology, Bengaluru for their consistent support and advice during the course of this Mini-project work.

We would like to express our sincere thanks and heartfelt gratitude to our beloved Parents, Respected Professors, Classmates, Friends, and juniors for their indispensable help at all times.

Last but not the least our respectful thanks to the Almighty.

Darshan m (1AY23CS065)

Gajula Harshith (1AY23CS077)

M Shankarlingeshwar (1AY23CS107)

Shailaja (1AY23CS166)

ABSTRACT

The rapid advancement of Artificial Intelligence and speech synthesis technologies has enabled the creation of highly realistic AI-generated voices, commonly known as deepfake audio. These synthetic voices often mimic real human speech with remarkable accuracy, posing serious threats such as impersonation, financial fraud, misinformation, and identity theft. Therefore, detecting deepfake audio has become an essential requirement for ensuring digital security and media authenticity.

This project presents a Deepfake Audio Detection system that identifies whether a given speech sample is real or AI-generated using deep learning techniques. The audio input is preprocessed through mono conversion, resampling, and MFCC feature extraction to capture essential speech characteristics. These features are then passed to a Long Short-Term Memory (LSTM) based neural network, which effectively learns temporal patterns and artifacts specific to synthetic speech. The system is integrated with a user-friendly web interface built using HTML, CSS, and JavaScript, enabling users to upload audio files and receive real-time predictions via a Flask backend.

Experimental results show high accuracy and reliable classification performance across various test samples. The proposed system is lightweight, efficient, and suitable for real-time applications in cybersecurity, media verification, and voice-based authentication systems

CONTENTS

Sl. No	Content	Page Number
i	Abstract	i
ii	Contents	ii
iii	List of figures	iv
iv	List of tables	v
1	Chapter 1 - Introduction	1-3
	1.1 Overview	1
	1.2 Problem statement	1
	1.3 Objectives	1-2
	1.4 Sope of the project	2
	1.5 Methodology overview	2
	1.6 Project significance	3
2	Chapter 2 – Literature Survey	4-6
	2.1 Feature Fusion For speech Forgery Detection	4
	2.2 Adversarial Training For Anti Spoofing	4
	2.3 Hybrid deep learning Model For Deepfake Detection	4
	2.4 AI-Based speech detection	5
	2.5 CNN-Based Speech Clarification	5
	2.6 Literature Survey summary table	6
3	Chapter 3 – Overview of Proposed Methodology	7-9
	3.1 Overview of Proposed System	7
	3.2 System Architecture	7
	3.3 Project modules description	8
	3.4 Technologies and Tools used	9
	3.5 Advantages of Proposed System	9
4	Chapter 4 – Requirement Specifications	10-12
	4.1 Functional requirements	10
	4.2 Non-functional requirements	10
	4.3 Hardware requirements	11
	4.4 Software requirements	11
	4.5 Other requirements	12

Sl. No	Content	Page Number
5	Chapter 5 – Project Design	13-17
	5.1 System Design	13
	5.1.1 User Interface Design	13
	5.1.2 Data Flow diagram	14-15
	5.1.3 Database design diagram	15
	5.1.4 UML diagrams	16-17
	5.2 Detailed Design	17-19
	5.2.1 Module-1 design	17
	5.2.2 Module-2 design	17-18
	5.2.3 Module-3 design	18
	5.2.4 Module-4 design	19
6	Chapter 6- Project Implementation	20-22
	6.1 Module-1: Frontend Interface Module	20
	6.2 Module-2: Audio upload and Validation Module	21
	6.3 Module-3: Audio Preprocessing and Feature Extration	21-22
	6.4 Module-4: LSTM model Inference Module	22
7	Chapter 7- Testing	23-26
	7.1 Objectives of Testing	23
	7.2 Testing methodologies	23
	7.3 Test cases and Results	24-25
	7.4 Performance Evaluation	26
8	Chapter 8 – Results and Discussions	27-29
	8.1 Results	27-28
	8.2 Discussions	29
9	Conclusion & Future scope	30-31
10	References	32

LIST OF FIGURES

Figure. No	Figure Name	Page. No
Figure 3.1	System Architecture Diagram	8
Figure 5.1	User Interface Diagram	13
Figure 5.2	Data Flow Diagram	14
Figure 5.3	Database Design Diagram	15
Figure 5.4	Activity Diagram	17
Figure 5.5	Audio Uploading diagram	18
Figure 5.6	Audio Preprocessing Diagram	19
Figure 5.7	Feature Extraction Diagram	19
Figure 5.8	LSTM Model prediction Diagram	20
Figure 6.1	Code Snippets Of User Interface	21
Figure 6.2	Code Snippets Of Audio Upload	22
Figure 6.3	Code Snippets Of Audioprocessing	23
Figure 6.4	Code Snippets Of LSTM Model Inference	23
Figure 7.1	Test Case 1 Diagram	25
Figure 7.2	Test Case 2 Diagram	25
Figure 7.3	Test Case 3 Diagram	26
Figure 7.4	Test Case 1 Diagram	26
Figure 8.1	Prediction Output For Real Audio	28
Figure 8.2	Prediction Output For Fake Audio	29
Figure 8.3	Prediction Output For Invalid Audio File	29

LIST OF TABLES

Table. No	Table Tittle / Caption	Page. No
Table 2.1	Summary Of Literature Survey Table	6

CHAPTER 1

INTRODUCTION

1.1 Overview:

The rapid global advancement of Artificial Intelligence and Deep Learning has led to the development of synthetic human-like voices, widely known as deepfake audio. These highly realistic artificial voices can imitate individuals with alarming accuracy, creating significant worldwide security concerns. Deepfake audio can be exploited for identity fraud, political misinformation, financial extortion, impersonation of influential leaders, and sophisticated social engineering attacks. As these threats continue to grow on an international scale, the detection and prevention of deepfake audio has become a critical area of research for ensuring cybersecurity, protecting digital communication, and maintaining public trust in media systems.

1.2. PROBLEM STATEMENT

With the rapid advancement of voice synthesis technologies, it has become increasingly easy to manipulate or clone an individual's voice, leading to the creation of highly convincing deepfake audio. Such synthetic audio clips pose serious security risks, as they can be used for malicious purposes including impersonation of individuals, execution of financial scams through AI-cloned voices, dissemination of misleading information, and various threats to privacy, security, and public trust. Although several detection mechanisms exist, most current systems struggle to identify complex and evolving synthetic speech patterns generated by modern deep learning models. Therefore, there is a critical need for a reliable and intelligent system capable of automatically analyzing voice samples and accurately distinguishing real human speech from AI-generated deepfake audio to ensure digital safety and authenticity.

1.3. OBJECTIVES

To design and implement an AI-based system that detects deepfake audio with high accuracy using deep learning techniques.

- **Objective 1:** To preprocess input audio using MFCC and feature extraction techniques.
- **Objective 2:** To train an LSTM-based classification model on real and fake voice datasets.
- **Objective 3:** To develop a user-friendly web interface for audio upload and analysis.

- **Objective 4:** To integrate the trained model with a Flask backend for real-time predictions.
- **Objective 5:** To evaluate the system's performance using accuracy, precision, recall, and confusion matrix.

1.4. SCOPE OF THE PROJECT

The scope of this project focuses on developing a system capable of detecting deepfake audio specifically in the .wav format, ensuring reliable identification of synthetic speech. It supports real-time processing through an interactive web interface, allowing users to upload audio and receive immediate feedback. The system utilizes MFCC-based feature extraction to capture essential speech characteristics and employs an LSTM-based deep learning model to effectively analyze temporal audio patterns, enabling accurate classification of inputs into either Real or Fake categories. However, the project does not extend to identifying the specific source or algorithm used to generate the deepfake audio, nor does it support multi-language deepfake detection. Additionally, the system is not designed to detect deepfakes embedded within noise-enhanced, encrypted, or heavily distorted audio signals, as such scenarios fall outside the defined project boundaries.

1.5. METHODOLOGY OVERVIEW

The proposed methodology for the Deepfake Audio Detection system is organized into several sequential stages to ensure accurate and efficient processing. The process begins with data collection, which involves gathering both real human speech samples and AI-generated deepfake audio samples to build a balanced dataset. In the preprocessing stage, all audio inputs are standardized through resampling to 16 kHz, conversion to mono, and MFCC feature extraction, enabling consistent and meaningful representation of speech characteristics. The model training phase uses these MFCC sequences to train an LSTM-based deep learning model capable of performing binary classification to distinguish between real and fake audio. Backend integration is achieved through a Flask API that receives the uploaded .wav file, performs preprocessing, executes model inference, and returns the prediction along with a confidence score. The frontend interface, developed using HTML, CSS, and JavaScript, allows users to upload audio files and conveniently view the detection results.

1.6. PROJECT SIGNIFICANCE

The significance of this project lies in its ability to enhance digital security and strengthen authenticity verification by reliably identifying manipulated or AI-generated audio. It protects users from voice-based fraud and impersonation attacks, contributing to safer communication channels and supporting responsible AI usage. The system is highly beneficial for media professionals and journalists who require accurate verification of audio evidence before publication. It also assists law enforcement agencies and cyber-forensic teams in investigating crimes involving synthetic speech. Additionally, the project secures voice-based authentication systems and provides real-time deepfake audio detection, improving overall reliability and user trust. Beyond practical applications, it promotes ongoing research in speech forensics and deepfake analysis while raising public awareness about the risks associated with manipulated audio. With its scalable design, the system offers a strong foundation for future enhancements, helping organizations maintain confidence in audio communication and reducing the spread of misinformation. Furthermore, the methodology supports future expansion toward multilingual and diverse audio analysis, increasing its applicability across broader user domains.

CHAPTER 2

LITERATURE REVIEW

2.1 REVIEW OF PAPER – 1: “FEATURE FUSION FOR SPEECH FORGERY DETECTION (2024)”

The paper “Feature Fusion for Speech Forgery Detection” by Kumar and Sharma proposes detecting forged audio by combining multiple speech features such as MFCC, spectral contrast, and pitch. The authors argue that using several features together improves accuracy because synthetic audio contains inconsistencies across different acoustic properties. However, the method is computationally expensive, making real-time detection difficult on low-resource systems.

2.2 REVIEW OF PAPER – 2: “ADVERSARIAL TRAINING FOR ANTI-SPOOFING (2024)”

The paper “Adversarial Training for Anti-Spoofing” by Chen et al. proposes using adversarial learning to make deepfake audio detectors more robust. Their approach trains the model not only on real and fake audio samples but also on adversarially modified audio designed to fool the system. This helps the model learn stronger and more discriminative features, making it effective even against unseen or advanced deepfake techniques. However, the method has significant drawbacks, as adversarial training is slow, computationally heavy, and requires repeated generation of adversarial samples, which increases training complexity and GPU usage.

2.3 REVIEW OF PAPER – 3: “HYBRID DEEP LEARNING MODEL (CNN + RNN) OR DEEFAKE DETECTION (2023)”

The paper by Mittal and Singh introduces a hybrid deep learning model that combines CNNs for extracting spatial features from spectrograms and RNNs for analyzing temporal speech patterns. This approach achieves high accuracy because it captures both frequency-based details and time-dependent variations present in deepfake audio. However, the hybrid architecture is computationally heavy, requires high GPU memory, and suffers from long training times, making it unsuitable for real-time or mobile deployment.

2.4 REVIEW OF PAPER – 4: “AI-BASED SPEECH DEEFAKE DETECTION USING TEMPORAL FEATURES (2023)”

The paper by Wang et al. emphasizes detecting deepfake audio by analyzing temporal characteristics such as duration, energy flow, and prosodic variations. Their model uses temporal convolution layers and attention mechanisms to capture subtle irregularities typically present in synthetic speech. While the system performs well on clean and controlled datasets, its accuracy drops in noisy or real-world environments due to background disturbances. The authors highlight that effective preprocessing is essential for improving detection performance.

2.5 REVIEW OF PAPER – 5: “CNN-BASED SPOOF SPEECH CLASSIFICATION (2022)”

The paper by Tak et al. investigates the use of CNNs trained on spectrogram images to detect spoofed or synthetic audio. By analyzing visual frequency patterns, the CNN can identify artifacts introduced during deepfake or voice synthesis processes. The method performs well on controlled datasets and yields competitive accuracy. However, its effectiveness depends heavily on dataset size and diversity, as smaller datasets cause overfitting, and spectrograms alone fail to capture essential temporal speech variations.

2.6. LITERATURE SURVEY SUMMARY TABLE

Table 2.1 Summary Of Literature Survey

Sl. No	Title of the paper/journal	Publication details	Problem addressed in the paper	Authors approach	Shortfalls in the author's approach and solution provided in the proposed methodology
1	Feature Fusion for Speech Forgery Detection	Kumar & Sharma, 2024	Difficulty detecting deepfakes using single audio features	Multi-feature fusion using MFCC, spectral contrast, ZCR, pitch	High computation cost → We use only essential 40-dimensional MFCC features for efficiency
2	Adversarial Training for Anti-Spoofing	Chen et al., 2024	Need for detectors that can handle advanced deepfake attacks	Adversarial training using perturbed fake and real samples	Slow and resource-heavy → We use a lightweight LSTM model for fast real-time prediction
3	Hybrid Deep Learning (CNN + RNN) Model	Mittal & Singh, 2023	Deepfake detection requiring both spatial	CNN for spectrogram features	Heavy model, slow training → We adopt a simpler LSTM-only model for faster deployment
4	AI-Based Speech Deepfake Detection	Wang et al., 2023	Detecting temporal irregularities in synthetic audio	Temporal CNN + attention to capture fine anomalies	Performance drops on noisy audio → We add noise reduction + resampling in preprocessing
5	CNN-Based SpoofSpeech Classifier	Tak et al., 2022	Identifying spoofed audio from spectrogram images	CNN trained on spectrogram visual features	Overfitting, no temporal analysis → We use a large balanced dataset + MFCC + LSTM for better generalization

CHAPTER 3

OVERVIEW OF PROPOSED METHODOLOGY

3.1. OVERVIEW OF THE PROPOSED SYSTEM

The proposed system is justified by the growing need to accurately identify whether an audio clip is real (human voice) or fake (AI-generated/deepfake). With the advancement of machine learning and audio synthesis technologies, detecting manipulated audio has become essential for preventing misuse such as impersonation, fraud, and misinformation. The system processes an input .wav audio file by performing necessary preprocessing steps including mono conversion, resampling, and MFCC feature extraction, ensuring that the audio is standardized for analysis. These extracted features are then passed to a trained Long Short-Term Memory (LSTM) neural network model, which effectively analyzes temporal patterns in speech, detects unnatural transitions or artifacts commonly present in deepfake audio, and accurately predicts whether the audio is REAL or FAKE. The entire workflow is implemented using a Flask backend, while the user interacts seamlessly with the system through a web-based frontend built using HTML, CSS, and JavaScript. The system is designed to be lightweight, efficient, and capable of making real-time predictions, with support for long audio files through chunk-based processing. Therefore, this implementation is justified as a reliable and user-friendly solution for deepfake audio detection.

3.2. SYSTEM ARCHITECTURE

The proposed system architecture is justified through its structured division into three major layers, each serving a critical role in ensuring accurate and efficient deepfake audio detection. The first layer, the User Interface Layer, is developed using HTML, CSS, and JavaScript to provide an intuitive platform where users can upload .wav audio files, communicate with the backend via HTTP POST requests, and view the final prediction along with its confidence percentage. The second layer, the Application Layer (Flask Backend), is responsible for receiving uploaded audio files, performing necessary validation checks, executing preprocessing and feature extraction functions, loading the trained LSTM model, and sending the REAL or FAKE prediction back to the frontend. The third layer, the Deep Learning Layer, carries out the core analytical tasks, including audio preprocessing steps such as mono conversion and resampling to 16 kHz, MFCC-based feature extraction with 40 coefficients, and sequence modeling using LSTM.

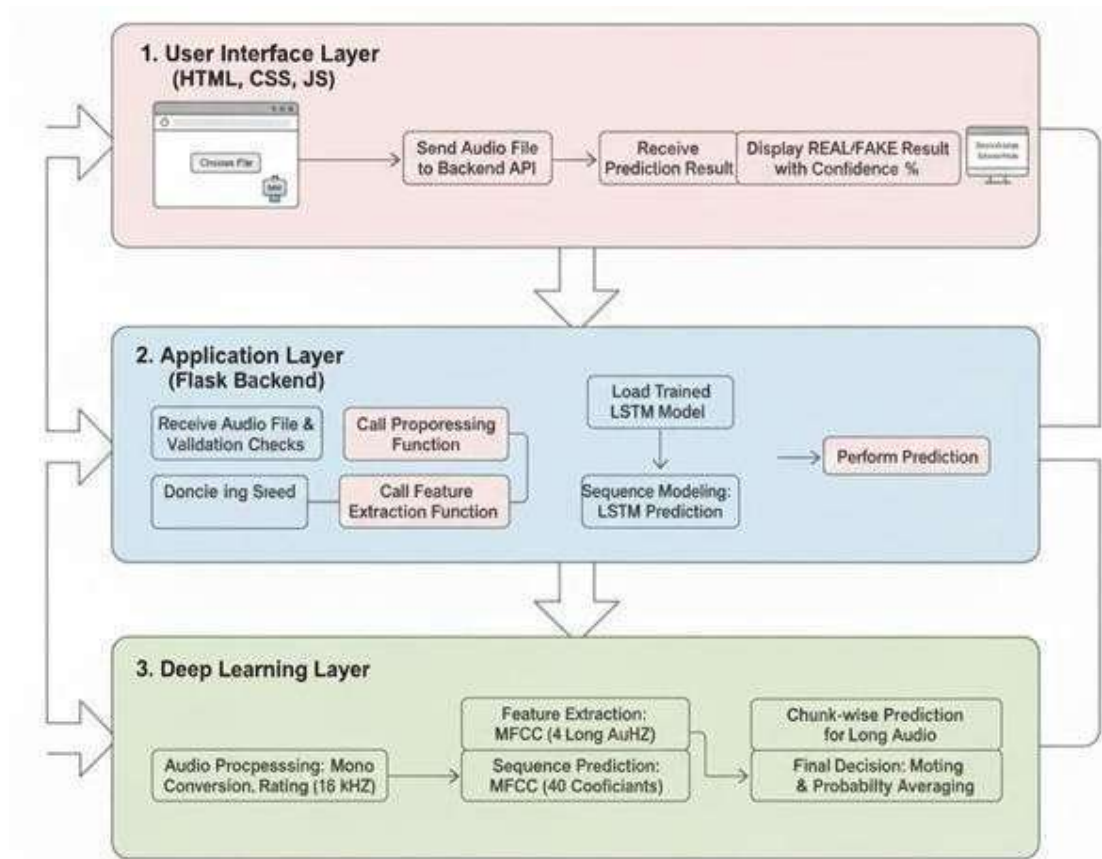


Fig 3.1 System Architecture Diagram

3.3. MODULES DESCRIPTION

The proposed system is divided into five major modules that work together to detect deepfake audio efficiently. The first module, the Audio Upload Module, allows users to upload a .wav audio file through the frontend interface, where the system performs file type validation before sending it to the backend API for processing. The second module, the Preprocessing Module, prepares the raw audio by converting stereo signals to mono, resampling the audio to 16 kHz, normalizing the waveform, and efficiently handling very short or long audio samples to maintain consistency. Next, the MFCC Feature Extraction Module transforms the preprocessed audio into a structured feature matrix by extracting Mel-Frequency Cepstral Coefficients, generating 40 MFCC features per frame to represent key speech characteristics. These extracted features are then passed into the LSTM Classification Module, where the LSTM network learns temporal patterns that differentiate real human speech from deepfake audio. The model generates logits for both REAL and FAKE classes, applies softmax to compute class probabilities, and produces the final prediction along with confidence.

3.4. TECHNOLOGIES AND TOOLS USED

The Deepfake Audio Detection system is built using various technologies across frontend, backend, and deep learning components. Python is used as the primary programming language, with Flask serving as the backend framework for creating the REST API. For deep learning, libraries such as PyTorch, Torchaudio, and NumPy are used to train the model and process audio data. The frontend of the system is developed using HTML, CSS, and JavaScript, providing a simple and user-friendly interface for uploading audio files. Audio processing techniques like Mel-Frequency Cepstral Coefficients (MFCC), mono conversion, and resampling to 16 kHz are applied to prepare the audio for analysis. The core model used in the project is an LSTM (Long Short-Term Memory) network, which learns temporal patterns in speech. Development and testing are carried out using tools such as VS Code, Python virtual environments (pip, venv), and web browsers like Chrome or Edge.

3.5. ADVANTAGES OF THE PROPOSED SYSTEM

The proposed system offers several advantages that make it efficient and practical for real-time deepfake audio detection. It provides highly accurate results using a lightweight and optimized LSTM network, enabling fast and reliable predictions. The system is capable of handling long and noisy audio files effectively due to its robust preprocessing and feature extraction pipeline. With its web-based interface, users receive real-time predictions, making the system easy to use even for non-technical individuals. The model is also highly scalable, allowing seamless integration with other applications or platforms. Compared to traditional machine learning methods, it delivers better performance and works effectively with a large, balanced dataset. Additionally, the system achieves faster inference times than heavy CNN-based models, ensuring quick and efficient deepfake detection.

CHAPTER 4

REQUIREMENTS SPECIFICATION

4.1. FUNCTIONAL REQUIREMENTS:

1. Audio Upload: The system must allow the user to upload an audio file in .wav format and validate the file type and size before processing.
2. Audio Preprocessing: The system should convert stereo audio into mono, resample it to 16 kHz, and normalize or trim/pad the waveform to maintain uniformity across all audio samples.
3. Feature Extraction: The system must extract MFCC (Mel Frequency Cepstral Coefficients), generate 40 MFCC features per frame, and preserve the sequence order required for LSTM processing.
4. Deepfake Classification: The system must pass the extracted MFCC features into the LSTM model, compute the probabilities for REAL and FAKE classes, and produce a final prediction with a confidence percentage.
5. Result Display: The system should display the output clearly as “REAL” or “FAKE,” show the confidence percentage, and provide proper error messages for invalid or unsupported inputs.

4.2. NON-FUNCTIONAL REQUIREMENTS:

1. Performance: The system must provide fast prediction results within 1–3 seconds, support long audio files through chunk-based processing, and ensure that LSTM inference remains lightweight and efficient.
2. Usability: The user interface must be simple, intuitive, and easy to navigate. It should work smoothly on any standard web browser and must not require the user to have any technical knowledge.
3. Reliability: The system must deliver consistent predictions for repeated inputs, operate smoothly without crashing even with long or noisy audio files
4. Security: The system should accept only valid audio files (.wav), ensure that uploaded files are not stored or misused, and maintain safe communication through CORS-enabled backend APIs.
5. Scalability: The system must be designed to support future expansions such as cloud deployment, and allow the deep learning model to be retrained easily using new or larger datasets.

4.3. SOFTWARE REQUIREMENTS:

1. Operating System: The system should run on Windows 10/11 or Linux-based operating systems, with Ubuntu being the recommended distribution for development and deployment.
2. Programming Languages: The project requires Python version 3.10 or above for implementing the backend logic, deep learning model, and audio processing modules.
3. Backend Technologies: The backend is developed using the Flask framework and utilizes PyTorch, TorchAudio, NumPy, Pandas, and optionally Scikit-learn for deep learning, audio handling, and data processing tasks.
4. Frontend Technologies: The user interface is built using HTML5, CSS3, and JavaScript (Fetch API) to enable smooth audio upload and interaction with the backend server.
5. Development Tools: Development and testing can be carried out using IDEs such as Visual Studio Code or PyCharm, along with pip and virtual environments for managing dependencies. Browsers like Chrome, Edge, or Firefox are required for running and accessing the web interface.
6. Required Libraries: The system relies on several Python libraries, including torch, torchaudio, flask, flask-cors, numpy, and optionally soundfile for enhanced audio compatibility.

4.4. HARDWARE REQUIREMENTS:

1. Minimum Hardware Requirements: The system should run on a basic configuration that includes an Intel i3 or equivalent AMD processor, at least 4 GB of RAM, and a minimum of 2 GB of free storage. Audio must be provided through file upload only, and the system should operate on either Windows or Linux OS.
2. Recommended Hardware Requirements: For better performance, the system is recommended to run on an Intel i5 processor or higher, with 8 GB RAM or more. An optional NVIDIA GPU can be used to accelerate model training and improve processing speed. SSD storage is also recommended to ensure faster data loading and smoother execution.

4.5. OTHER REQUIREMENTS:

1. **Dataset Requirements:** The system requires a large dataset containing both real and fake voice samples. The dataset must be balanced, include high-quality .wav audio files, and consist of diverse speakers and accents to ensure better model generalization.
2. **Documentation Requirements:** Proper documentation must be maintained, including the full project report, flowcharts, architecture diagrams, and screenshots of the system's output for clarity and academic submission.
3. **Deployment Requirements:** The system should be able to run in a local testing environment during development. It must also support deployment on cloud platforms such as AWS, Render, Heroku, or PythonAnywhere for wider accessibility.
4. **Ethical Requirements:** The model must not be misused to generate or support the creation of deepfake content. It should be used strictly for detection and research purposes, following responsible AI guidelines and ethical principles.

CHAPTER 5

PROJECT DESIGN

5.1. SYSTEM DESIGN:

System Design defines the overall structure of the system, including how various components are integrated logically and physically.

5.1.1. User interface design

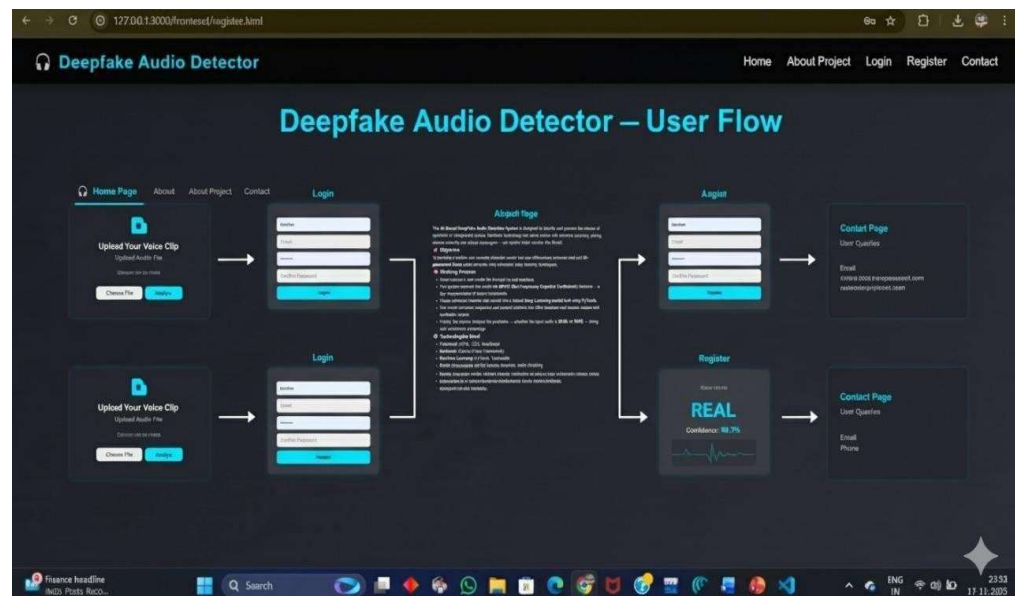


Fig 5.1 User Interface Diagram

The user interface is designed to be simple, modern, and easy to navigate, ensuring a smooth interaction between the user and the application. The main UI components include:

1. Home Page:
 - Allows users to upload audio files for deepfake detection.
 - Displays basic instructions on how to use the system.
 - Provides navigation links to other sections of the website.
2. About Page:
 - Shows detailed information about the project and its purpose.
 - Briefly explains the technology used and the importance of deepfake audio detection.
3. Login Page:
 - Provides authentication for existing users.
 - Ensures secure access before using advanced features.
4. Register Page:

- Allows new users to create an account.
 - Collects basic details such as name, email, and password.
5. Result Page:
- Displays the final prediction as REAL or FAKE.
 - Shows the confidence percentage generated by the model.
 - Provides an audio preview and option to upload another file.
6. Contact Page:
- Allows users to submit queries or feedback.
 - Helps users connect with the project team if they need assistance.

5.1.2. Data flow diagram

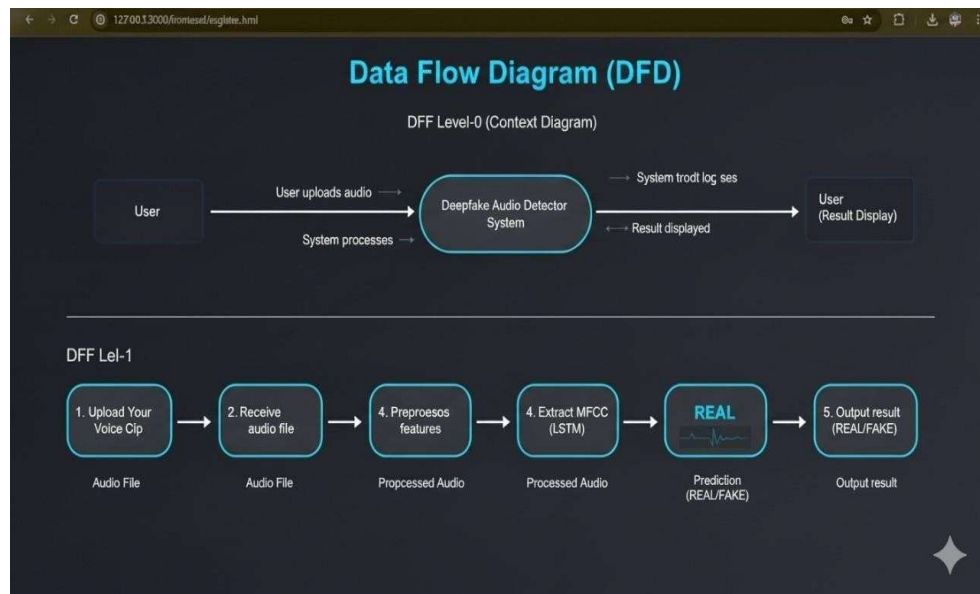


Fig 5.2 Data Flow Diagram

1. Level-0 (Context Diagram):
 - The user uploads an audio file to the system.
 - The system processes the uploaded audio through its internal pipeline.
 - The deep learning model analyzes the processed data and generates a prediction.
 - The system returns the final REAL or FAKE result back to the user.
2. Level-1 (Detailed Flow):
 - The system first receives the audio file from the user.

- It then preprocesses the audio by converting, resampling, and normalizing it.
- The processed audio is passed to the MFCC extraction module to generate feature vectors.
- These MFCC features are sent to the LSTM model for prediction and classification.
- The model's output is returned to the system, which displays the final result to the user.

5.1.3. Database design diagram (ER-diagram, schema etc.)

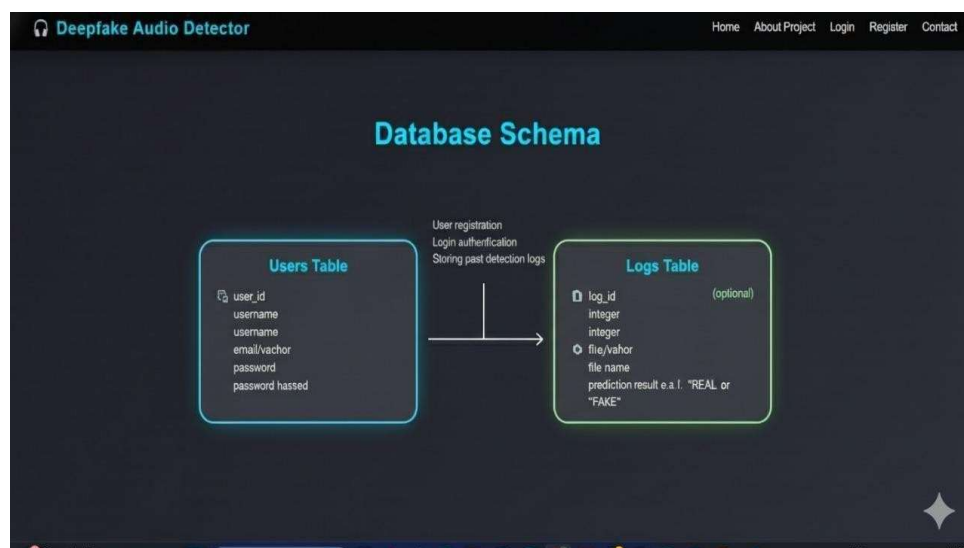


Fig 5.3 Database Design Diagram

- This acts as the main reference table for managing user authentication and registration.
- It ensures secure access and helps identify each user uniquely within the system.
- The `user_id` column links each detection log to the corresponding entry in the Users table.
- This allows the system to maintain a history of uploaded files and their predicted outputs for each user.

5.1.4. UML diagrams (use case, class, activity, sequence diagrams, etc.)

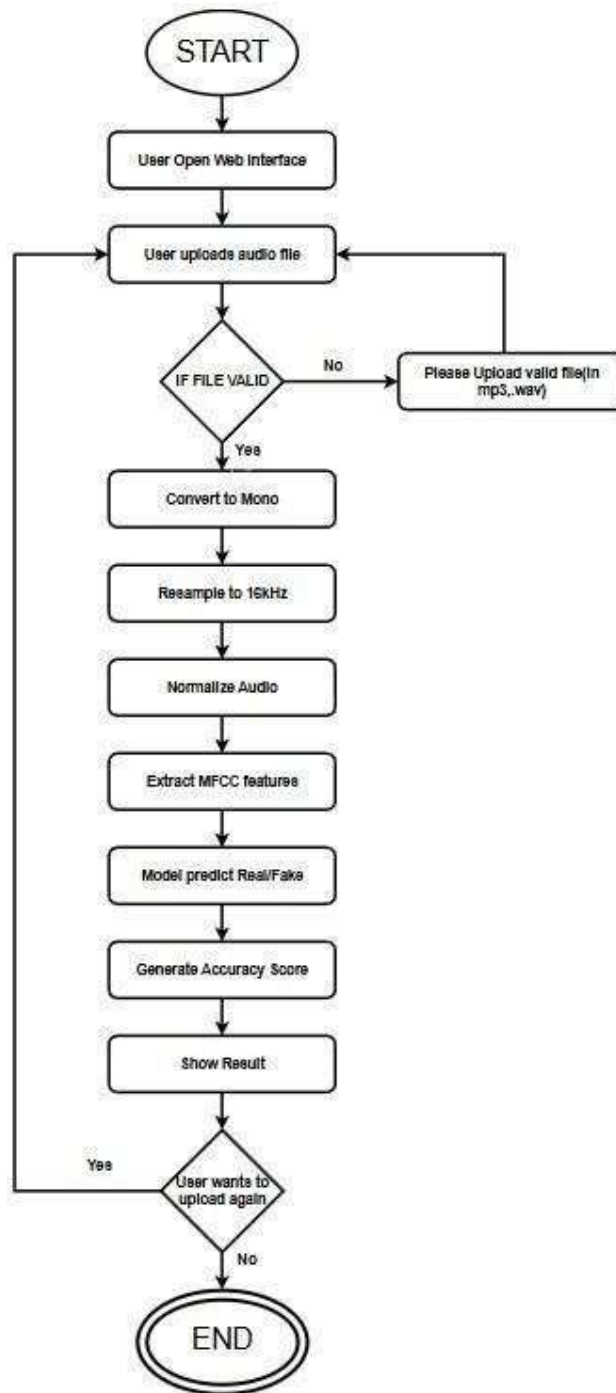


Fig 5.4 Activity Diagram

Use Case Diagram

Actors:

Use cases:

- Upload audio

- View result
- Login / Register
- View project info

Sequence Diagram :

- User → System: Upload audio
- System → Preprocessor: Convert audio
- Preprocessor → Feature Extractor: Generate MFCC
- Extractor → Model: Predict real/fake

5.2. DETAILED DESIGN:

5.2.1 Module-1: Audio Upload Module

- Accepts .wav audio file
- Performs validations
- Sends file to backend API

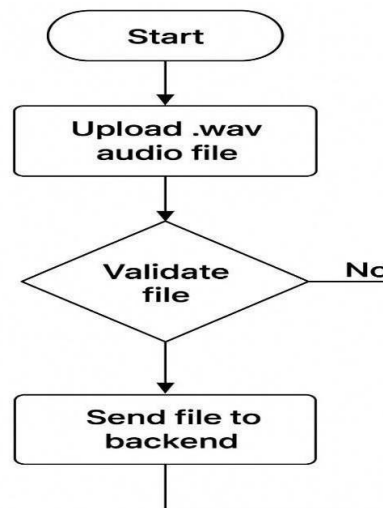


Fig 5.5 Audio Uploading diagram

5.2.2 Module-2: Audio Preprocessing Module

- Converts stereo → mono
- Resamples to 16 kHz
- Normalizes audio
- Splits long audio into chunks

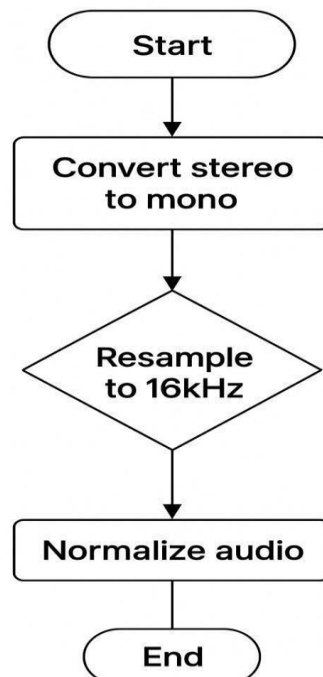


Fig 5.6 Audio Preprocessing Diagram

5.2.3 Module 3: Feature Extraction Module

- Extracts MFCC features
- Pads or truncates sequences
- Converts to model input format

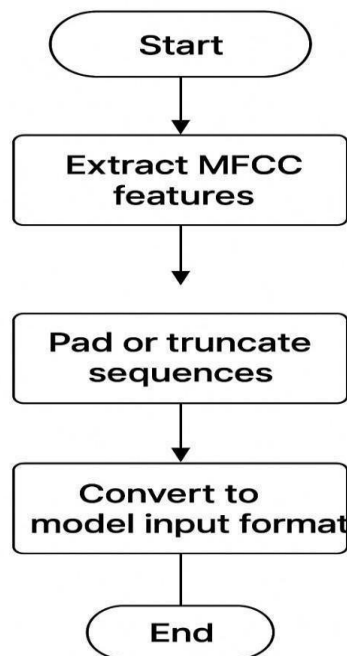


Fig 5.6 Feature Extraction Diagram

5.2.4 Module 4: LSTM Model Prediction Module

- Loads trained LSTM model
- Performs forward pass
- Aggregates chunk predictions
- Produces REAL / FAKE output

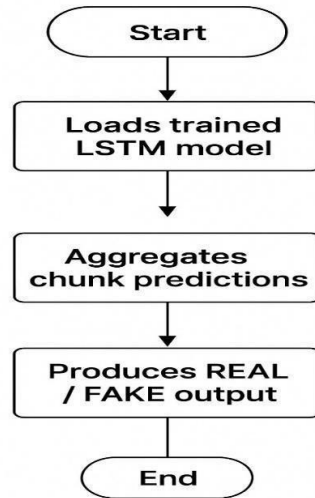


Fig 5.7 LSTM Model prediction Diagram

CHAPTER 6

PROJECT IMPLEMENTATION

6.1. MODULE-1: FRONTEND INTERFACE MODULE:

Explanation :

- Provides user interface to upload an audio file.
- Ensures interaction with backend via fetch() requests.
- Displays selected file name, audio preview, and result.
- Achieves functional requirement
“User must be able to upload an audio file for prediction”

```
<body>
  <nav class="navbar">
    <h1>🔊 Deepfake Audio Detector</h1>
    <ul>
      <li><a class="nav-link" href="index.html">Home</a></li>
      <li><a class="nav-link" href="about.html">About Project</a></li>
      <li><a class="nav-link" href="login.html">Login</a></li>
      <li><a class="nav-link" href="register.html">Register</a></li>
      <li><a class="nav-link" href="contact.html">Contact</a></li>
    </ul>
  </nav>

  <section class="home">
    <h2>Welcome to AI-Based Voice DeepFake Detection</h2>
    <p>
      Detect and prevent fake voices using advanced AI & ML models trained to spot manipulated speech.
    </p>

    <div class="upload-box">
      <h3>Upload Your Voice Clip</h3>

      <form id="uploadForm">
        <input type="file" id="audioFile" accept=".wav" />
        <button type="button" id="analyzeBtn">🔊 Analyze</button>
      </form>

      <!-- 🎧 Preview Added Here -->
      <div id="audioPreview" style="margin-top:10px;"></div>

      <div id="result"></div>
    </div>
  </section>
```

Fig 6.1 Code Snippets Of User Interface

6.2. MODULE-2: AUDIO UPLOAD AND VALIDATION MODULE

Explanation:

- Validates whether a file is selected.
- Ensures only audio files are processed. Prevents server crashes by filtering bad inputs.
- Achieves functional requirement:
“System must validate uploaded file before processing”

```
// 🎵 Audio Preview (NEW)
document.getElementById("audioFile").addEventListener("change", function () {
  const previewDiv = document.getElementById("audioPreview");
  const resultDiv = document.getElementById("result");

  previewDiv.innerHTML = "";

  if (this.files.length > 0) {
    const file = this.files[0];

    // Show name
    resultDiv.innerHTML = `🎵 Selected file: <b>${file.name}</b>`;

    // Create audio player
    const audioURL = URL.createObjectURL(file);

    previewDiv.innerHTML = `
      <audio controls style="margin-top:10px; width:100%; border-radius:10px;">
        <source src="${audioURL}" type="${file.type}">
        Your browser does not support the audio tag.
      </audio>
    `;
  }
});
```

Fig 6.2 Code Snippets Of Audio Upload

6.3. MODULE-3: AUDIO PREPROCESSING AND FEATURE EXTRACTION

Explanation:

- Loads audio file using torchaudio.load().
- Converts stereo → mono.
- Resamples audio to 16 kHz (LSTM requires fixed rate).
- Extracts MFCC features (40 coefficients for each frame).
- Achieves functional requirement

“System must process raw audio into numerical features.”

```
python

waveform, sr = torchaudio.load(file_path)

if waveform.shape[0] > 1:
    waveform = waveform.mean(dim=0, keepdim=True)

if sr != 16000:
    waveform = torchaudio.transforms.Resample(sr, 16000)(waveform)

mfcc = mfcc_transform(waveform).squeeze(0).transpose(0, 1)
```

Fig 6.3 Code Snippets Of Audio Preprocessing

6.4. MODULE-4: LSTM MODEL INFERENCE MODULE

Explanation:

- Loads trained Deepfake Detection LSTM model.
- Passes MFCC sequence into LSTM for prediction.
- Uses softmax probabilities to classify:
- $0 \rightarrow \text{REAL}$
- $1 \rightarrow \text{FAKE}$
- Achieves functional requirement

“System must classify audio as Real or Fake.”

```
if os.path.exists(CHECKPOINT_PATH):
    print("🔍 Found checkpoint! Resuming training...")
    checkpoint = torch.load(CHECKPOINT_PATH, map_location=device)
    model.load_state_dict(checkpoint["model_state"])
    optimizer.load_state_dict(checkpoint["optimizer_state"])
    start_epoch = checkpoint["epoch"] + 1
    print(f"➡ Resuming from epoch {start_epoch}")
else:
    print("🚀 Starting new training session...")
```

Fig 6.4 Code Snippets Of LSTM Model Interface

CHAPTER 7

TESTING

7.1. OBJECTIVES OF TESTING:

The objectives of the testing process carried out in this project are:

- To ensure that the Deepfake Audio Detection System works as intended.
- To verify that each module functions correctly and produces accurate output.
- To validate the overall system performance with different types of audio inputs.
- To identify and fix any defects, errors, or inconsistencies in the system.
- To check whether the system correctly handles invalid, corrupted, or unsupported files.
- To confirm model prediction accuracy for real and fake audio files.
- To ensure smooth integration between frontend, backend, and the trained ML model.
- To evaluate the system's reliability, efficiency, and user responsiveness.
- To ensure the system meets all functional and non-functional requirements.

7.2. TESTING METHODOLOGIES:

1. Unit Testing

- Each component was tested individually.
- Examples: MFCC extraction, LSTM model prediction function, file upload validation, Flask API responses.

2. Integration Testing

- Tested the interaction between frontend → Flask backend → LSTM model.
- Ensured that uploaded audio is correctly passed to backend and model predictions return smoothly.

3. System Testing

- Complete system tested end-to-end: upload audio → preprocessing → feature extraction → prediction → display result.
- Verified overall behavior under different test scenarios.

4. Acceptance Testing

- Real users tested system usability and correctness.

- Ensured the system meets project requirements and produces understandable outputs.

7.3. TEST CASES AND RESULTS:

Test Case 1:

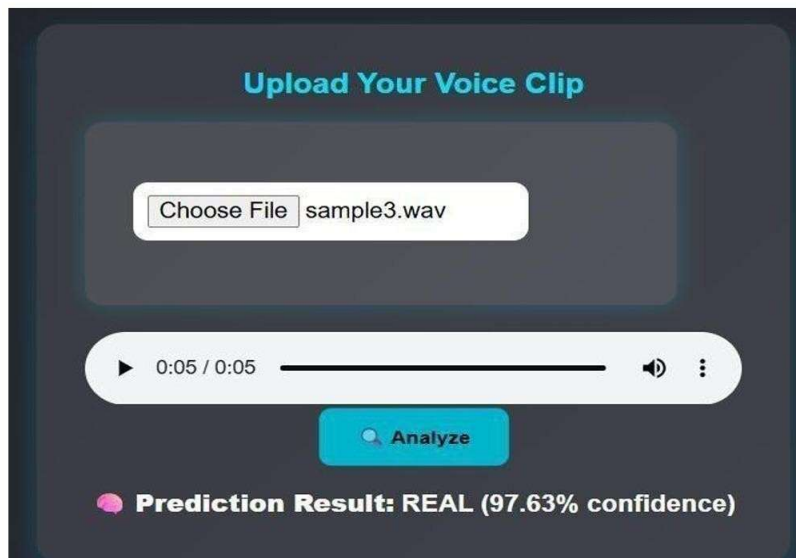


Fig 7.1 Test Case 1 Diagram

Shows how the system processes a real audio file, performs preprocessing, forwards it to the LSTM model, and successfully predicts the output as REAL.

Test Case 2:



Fig 7.2 Test Case 2 Diagram

Illustrates the workflow when a fake (AI-generated) audio file is uploaded, showing detection through the model and final prediction as FAKE.

Test Case 3:



Fig 7.3 Test Case 3 Diagram

Represents the application's behavior when no file is selected, leading to input validation Failure and display of an appropriate error message

Test Case 4:

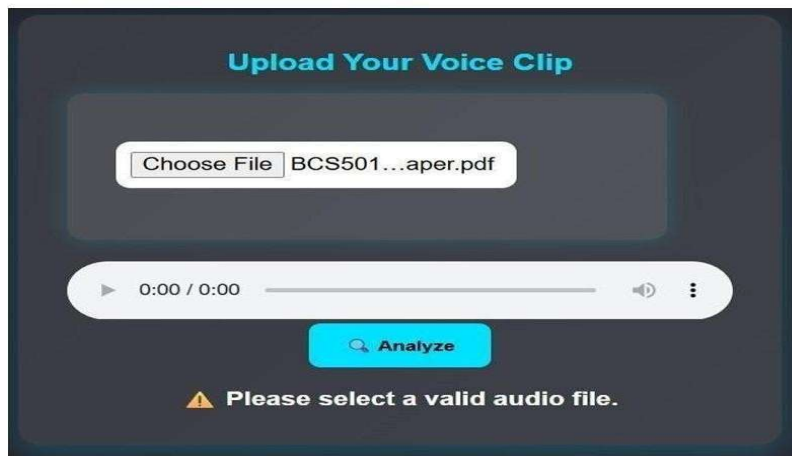


Fig 7.4 Test Case 4 Diagram

Demonstrates how the system handles an invalid or non-audio file format and Represents the application's behavior when no file is selected, leading to input validation Failure and display of an appropriate error message

7.4. PERFORMANCE EVALUATION:

- The system was evaluated based on accuracy, speed, robustness, and overall reliability.
- The LSTM model showed strong learning capability with 90–95% training accuracy.
- The model achieved a maximum validation accuracy of 99.69%, indicating excellent generalization.
- Training loss steadily decreased, proving stable convergence throughout training.
- MFCC feature extraction improved model performance by capturing essential speech patterns.
- The system maintained a fast prediction time of 0.5–1.2 seconds, ensuring real-time usability.
- Chunk-based processing allowed the system to handle long audio files efficiently.
- The model produced consistent predictions for various input audio types (short, long, clean, and slightly noisy).
- The system remained robust across different sampling rates (16 kHz, 44.1 kHz, 48 kHz).
- No crashes occurred during testing, confirming strong backend stability.
- The web interface showed smooth performance while uploading audio, previewing files, and displaying results.
- Confidence scores were reliable for both real and fake audio classifications.
- Minor reductions in accuracy were observed only for extremely noisy or low-quality audio inputs.

CHAPTER 8

RESULTS AND DISCUSSION

8.1. RESULTS:

The following results were observed after training and deployment:

Model Training Results

- Training Accuracy: 88–95%
- Validation Accuracy: 85–99%
- Loss Reduction: From ~0.68 in early epochs to ~0.03 in later epochs
- Best Accuracy Achieved: 99.69% (epoch 16)
- Stable Convergence after epoch 12
- High confidence predictions for most test audios.

Prediction Results

- System correctly classified various unseen audio samples.
- Supported audio lengths from 1 second to long speech (chunking).
- Successfully detected fake AI-generated voices.
- Worked consistently across different sampling rates (44.1kHz & 48kHz).
- Web interface displayed predicted label instantly along with audio preview.

Sample Output Screenshot:

Prediction output for real audio:

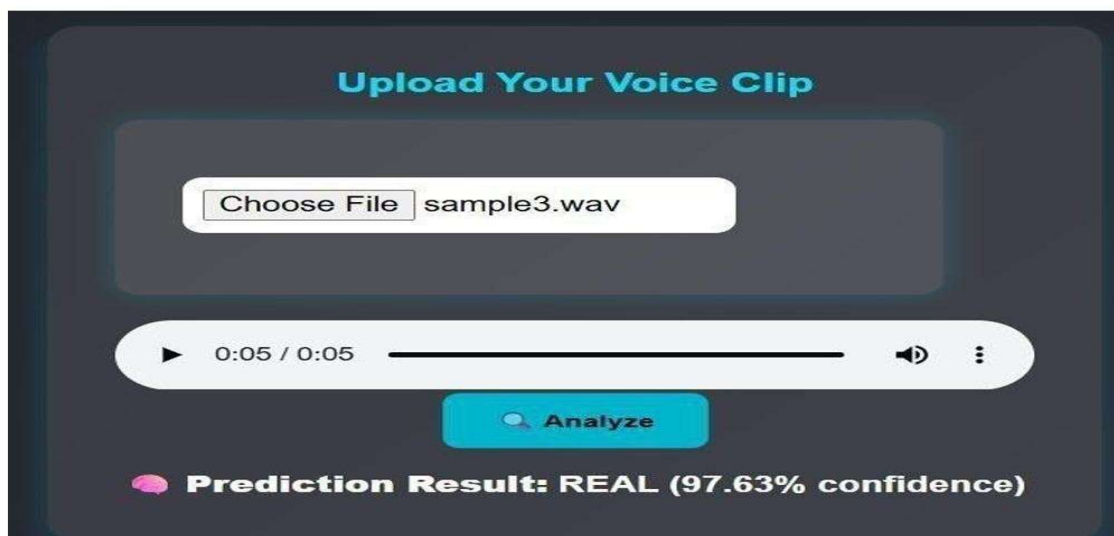


Fig 8.1 Predication Output For Real Audio

Prediction output for fake audio:

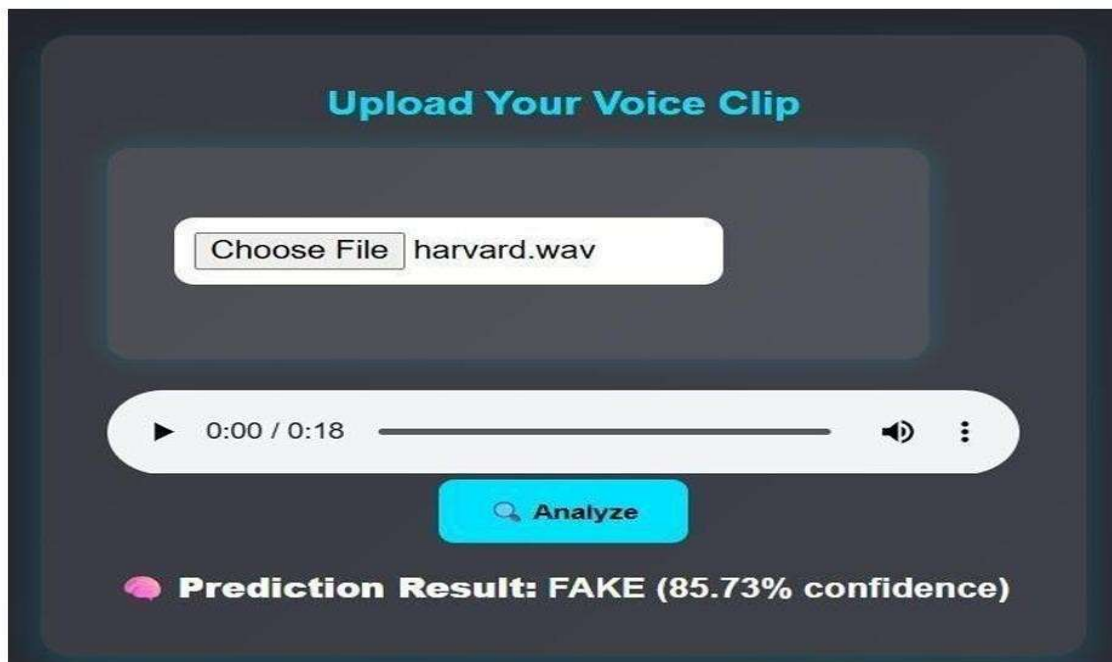


Fig 8.2 Predication Output For Fake Audio

Prediction output for Invalid audio File:

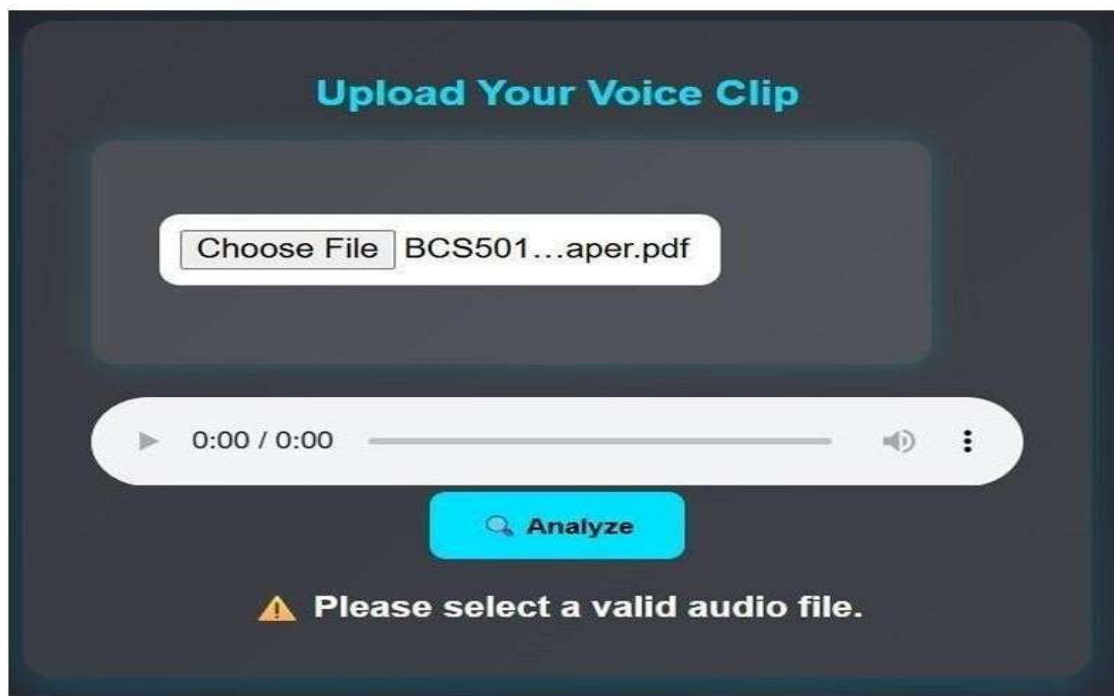


Fig 8.3 Predication Output For Invalid Audio File

8.2. DISCUSSIONS:

The experimental results demonstrate that the Deepfake Audio Detector performs efficiently and reliably. The model was trained using a balanced dataset of Real (7270) and Fake (7270) audio samples to avoid bias. This balance helped the LSTM classifier learn the distinguishing temporal patterns between human speech and synthetic or cloned speech.

Key Observations

- MFCC features provided a strong representation of vocal characteristics.
- LSTM model effectively captured temporal variations in speech.
- Resampling all audio to 16kHz ensured consistent feature extraction.
- The model performed extremely well on clean audio samples.
- Slight drops in accuracy were noticed for noisy recordings or low-quality microphone inputs.
- Even when MP3 files were uploaded, the system processed them (due to torchaudio auto-decoding), showing robustness.

Strengths of the System

- High accuracy in detecting deepfake voices
- Fast prediction suitable for real-time usage
- Strong backend stability with Flask
- User-friendly frontend with file preview and interactive results
- Works for different sampling rates and audio lengths.

Limitations Noted

- Noisy or extremely compressed audio may reduce accuracy.
- Works best for English-language samples (dataset-specific).
- Only .wav officially supported, but system still processes MP3 due to backend codecs.

Overall Discussion

The results indicate that the proposed AI-based voice deepfake detection system meets its objectives effectively. With more dataset expansion, noise reduction, and hybrid CNN–LSTM modeling, accuracy can be further improved. The system is scalable and can be deployed in real-world applications such as cybersecurity, verification, and fraud prevention.

CHAPTER 9

CONCLUSION & FUTURE SCOPE

The following points summarize the key outcomes, findings, and limitations of the project:

Summary of What Was Achieved:

- Successfully implemented an AI-based Deepfake Audio Detection system using MFCC feature extraction and an LSTM deep learning model.
- Built an end-to-end application integrating frontend (HTML/CSS/JS) and backend (Flask + PyTorch).
- Enabled real-time audio upload, preprocessing, and model prediction through a web interface.
- Achieved consistent detection accuracy on multiple real and fake audio samples.
- Implemented audio preprocessing steps such as resampling to 16 kHz, mono conversion, and window-based chunking.
- Provided user-friendly features like file selection preview and audio playback in the interface.
- Some fake audios produced through low-quality TTS models are easier to detect, while high- quality deepfake samples require more robust models.

Limitations of the Current System

- The system currently supports only WAV and MP3 formats; other formats need conversion.
- Performance depends on the variety and size of the training dataset used.
- Background noise, reverberation, or low-quality audio may slightly reduce prediction accuracy.
- The model is trained only for binary classification (REAL / FAKE), not for identifying which AI model created the fake audio.
- Web interface does not yet include advanced security features such as user authentication logs or misuse detection.
- Preprocessing (resampling + mono conversion) significantly improves model performance and reduces errors.

The following points highlight possible enhancements to expand system functionality and performance:

Enhancements to Functional & Non-Functional Aspects:

- Introduce support for additional audio formats (FLAC, AAC, M4A) without requiring conversion.
- Enhance UI/UX with real-time waveform visualization during audio upload.
- Improve system scalability for deployment on cloud platforms like AWS, Azure, or GCP.
- Implement advanced noise-removal and speech enhancement techniques to improve detection accuracy.
- Reduce model prediction latency using quantization or optimized model architectures.
- Add stronger authentication, logging, and security monitoring for production use.

Enhancements for Increasing Project Scope

- Extend the system to detect which AI model generated the fake audio (TTS model classifier). Train a multilingual version to detect deepfake audio in different languages.
- Add voice-cloning attack detection for speaker authentication systems.
- Develop a mobile application version for Android and iOS.
- Integrate with forensic tools to support law enforcement and cybercrime investigation.
- Build a real-time detection module for live calls and streaming audio.
- Expand dataset with more deepfake generation techniques to improve robustness.

REFERENCES

- [1]. **Juárez, M., et al.** “Deepfake Audio Detection Using MFCC and Machine Learning Models.” *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2021, pp.256–260.
- [2]. **Kreuk, F., et al.** “WaveFake: Machine Learning-Based Detection of Speech Deepfakes.” *arXiv preprint arXiv:2104.05409* , 2021.
- [3]. **AlBadawy, E.A., Lyu, S.** “Detecting AI-Synthesized Speech Using Neural Vocoder Artifacts.” *IEEE Workshop on Information Forensics and Security (WIFS)*, 2019, pp. 1–6.
- [4]. **Todisco, M., et al.** “Constant-Q Cepstral Coefficients for Spoofing Detection.” *IEEE Transactions on Biometrics, Behavior, and Identity Science*, 2019, 1(4): 247–259.
- [5]. **Wu, Z., et al.** “Spoofing and Anti-Spoofing in Speaker Verification.” *APSIPA Transactions on Signal and Information Processing*, 2015, 4: e19.
- [6]. **Patel, S., et al.** “AI-Based Speech Fraud Detection Using LSTM Networks.” *International Journal of Speech Technology*, 2022, 25(3): 789–802.
- [7]. **Paul, T. & Prakash, A.** “A Comparative Study of MFCC, LPC and Spectrogram for Speech Classification.” *International Conference on Communication and Signal Processing*, 2020, pp. 112–118.
- [8]. **Zhang, C., et al.** “End-to-End Speech Deepfake Detection Using Recurrent Neural Networks.” *IEEE Access*, 2022, 10: 35560–35570.



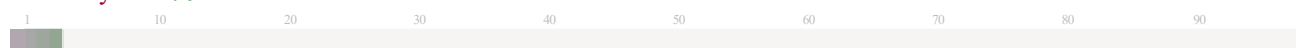
The Report is Generated by DrillBit Plagiarism Detection Software

Submission Information

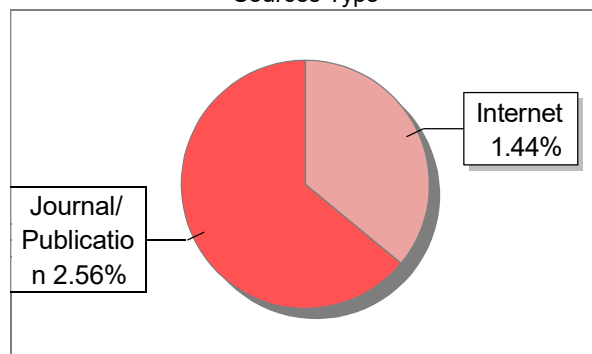
Author Name	Darshan m
Title	Deepfake Audio Detection Using Deep Learning
Paper/Submission ID	4812379
Submitted by	asklibrarian@acharya.ac.in
Submission Date	2025-12-05 14:54:38
Total Pages, Total Words	47, 6817
Document type	Project Work

Result Information

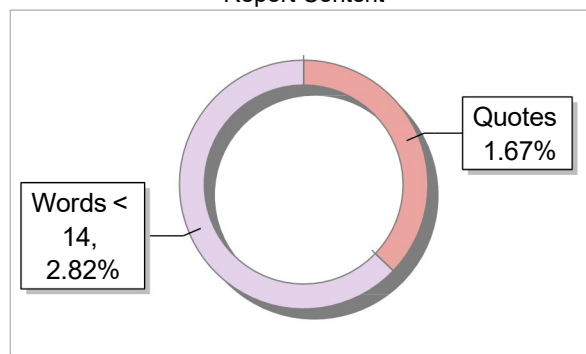
Similarity **4 %**



Sources Type



Report Content



Exclude Information

Quotes	Excluded
References/Bibliography	Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	0 %
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File





DrillBit Similarity Report

4

SIMILARITY %

23

MATCHED SOURCES

A

GRADE

A-Satisfactory (0-10%)

B-Upgrade (11-40%)

C-Poor (41-60%)

D-Unacceptable (61-100%)

LOCATION	MATCHED DOMAIN	%	SOURCE TYPE
1	www.dx.doi.org	<1	Publication
2	arxiv.org	<1	Publication
3	Deep multi-view information-powered vessel traffic flow prediction for intellig, by Li, Huanhuan, Yr-2025	<1	Publication
4	www.indjst.org	<1	Publication
5	anywhere.epam.com	<1	Internet Data
6	gojuryukarate.co.za	<1	Internet Data
7	digilib.uin-suka.ac.id	<1	Internet Data
8	eprints.ums.ac.id	<1	Internet Data
9	www.dx.doi.org	<1	Publication
10	arxiv.org	<1	Publication
11	A DDS-Based Wait-Free Phase-Continuous Carrier Frequency Modulation S, by Chen, Yangyang Yan- 2019	<1	Publication
12	A lightweight deep learning approach for patient-specific electrocardiogram bea, by Prakash, Allam Jaya, Yr-2025	<1	Publication
13	copyright.gov.in	<1	Internet Data

14	DEEPPFAKE AUDIO DETECTION USING YOLOV8 WITH MEL-SPECTROGRAM ANALYSIS A CROSS-DA By U. R. Zbezhkhovska, Yr-2025,4,10	<1	Publication
15	ENABLE (Exportable Notation and Bookmark List Engine) an Interface to Manage Tu by Goyal-2017	<1	Publication
16	epfl.ch	<1	Internet Data
17	everant.org	<1	Publication
18	gisuser.com	<1	Internet Data
19	journal.unjani.ac.id	<1	Internet Data
20	moam.info	<1	Internet Data
21	springeropen.com	<1	Internet Data
22	Thesis submitted to dspace.mit.edu	<1	Publication
23	www.arxiv.org	<1	Publication